# Instructor Handout 2: Computer-System Organization and Architecture

Auqib Hamid Lone
*Department of Computer Science & Engineering, GCET Ganderbal*

October 17, 2026

**Instructor Copy — Not for Student Distribution**

Concepts That Click

## 2.1 Before You Teach This Lecture

- **Silberschatz, Galvin & Gagne, *Operating System Concepts*, 8e** — Ch. 1 §1.2 (computer-system organization, interrupts, storage, I/O, DMA, pp. 6–18), §1.3 (computer-system architecture, pp. 15–20), and §1.7–1.8 (memory and storage management, pp. 24–26). *Page-verified.* This is your primary anchor: it is the only source that covers all five questions in this lecture in one place.
- **Stallings, *Operating Systems: Internals and Design Principles*, 6e** — Ch. 1 §1.4 (interrupts, p. 15), §1.5 (memory hierarchy, p. 26), §1.6 (cache, p. 29), §1.7 (I/O techniques and DMA, p. 33). *Page-verified.* Read these four sections for the engineering framing: Stallings is better than Silberschatz at showing *how* the interrupt cycle is wired, and his cache treatment is the one to borrow language from.
- **Bovet & Cesati, *Understanding the Linux Kernel*, 3e** — Ch. 4 (interrupts and exceptions, pp. 131–188). Optional. Read the first twenty pages if you want the concrete Linux vocabulary (top half, bottom half, softirqs) to sound natural rather than recited.
- **Rehearse the running thread once.** Write the five-step Enter-key trace from memory, in order, before class. The trace is the spine of the lecture and every later section hangs off it. If you cannot reproduce it without notes, re-read Silberschatz §1.2 once more.
- **Choose one machine to name.** Pick a laptop or lab machine the students actually use, and refer to its keyboard controller, its disk, and its core count when the abstract diagram needs an anchor.
- **Reserve the board in three zones.** Left: the anatomy and the interrupt path (drawn once, kept all hour). Centre: the running thread and the worked arithmetic. Right: vocabulary and the five-question roadmap. You will erase the centre repeatedly; do not erase the left zone.
- **Prerequisite check.** Students saw CPU, registers, memory, and interrupts in CS401. Spend at most two minutes recalling the memory hierarchy and the idea of an interrupt; do not re-

teach them. If the room is silent on "what is a register", that is your signal to slow down, not to lecture for ten minutes.

**Teaching note:** Read Silberschatz §1.2 for the mechanisms and Stallings §1.5–1.7 for the measured, engineering tone. Do not read all forty pages: the lecture is a mechanism lecture, not a survey. Your preparation time is better spent deriving the three worked examples on paper so that the arithmetic is automatic at the board.

## 2.2 Opening: The Enter-Key Thread

Write the command `cat bigfile.txt` on the board, large, and leave a blank line under it. Then say: "I press Enter. Before one character appears on the screen, name every component that had to do something." Collect four to six answers and write them around the command — keyboard, CPU, memory, disk, bus, screen. Do not correct anything yet. The mess is the point.

Then ask the sharper question: *which of these can signal the CPU, and how?* Most rooms will say the CPU "checks" the devices, or that the OS "watches" the keyboard. That answer is not stupid — it is polling — and it is exactly the design the lecture will replace. Write *poll* on the board next to the guess, and promise to price it later.

Put the five-question roadmap on the right side of the board, one line each: (1) what is the machine made of; (2) how does hardware get the CPU's attention; (3) where do data live; (4) how do bytes move; (5) how does the machine scale. Leave it visible for the whole hour; students use it to place each section.

Finally, state the promise of the thread: "We will follow this one command from the Enter key to the disk platter and back. An interrupt starts it, a storage hierarchy feeds it, and DMA moves the bytes." Nothing else needs to be resolved in the first five minutes.

**Teaching note:** Resist defining "interrupt" here. If a student offers the word, write it on the board and say only "we will earn that word in twenty minutes." The definition lands far better after they have felt the polling alternative fail. Six minutes of opening is the right budget; ten is too many.

**Board:** the opening itself is the first drawable moment. Build order: write the command first; collect components as a ring around it; underline the bus last and label it "shared". Narration for each stage: "this is the program", "these are the things it touches", "this is the thing they all touch". The last label is what the rest of the lecture is about.

## 2.3 Computer-System Anatomy and Bootstrap

Start with the structural claim, not the diagram: a computer is not a CPU that does everything. It is a set of independent devices that run concurrently and communicate over a shared bus. Then draw Figure 2.1.

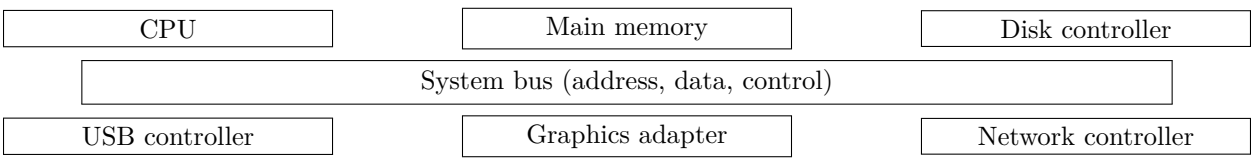



Figure 2.1: Anatomy of a computer system: independent devices attached to a shared system bus.

Concepts That Click

**Board:** draw the bus as a long horizontal channel first, not as a single wire — three groups of lines travel in it (address, data, control), and students who draw one line later cannot explain why the bus is a contended resource. Second, place CPU, main memory, and disk controller above it. Third, place the USB controller, graphics adapter, and network controller below it. Label the bus last, with all three groups. The narration: "CPU on the left, memory in the middle, the disk's agent on the right; everything talks through the channel." Ask, before labelling: "why is this drawn as a wide box rather than an arrow?" The answer is that several kinds of signals share it, and that sharing is why it can become a bottleneck.

**Definition (Device controller).** *Hardware that speaks the bus protocol and operates a physical device. A controller exposes registers the CPU can read and write, and it runs concurrently with the CPU.*

The word "controller" is doing real work: it is not the device itself. The disk controller is the electronics that positions the arm and streams sectors; the platter is the device. The same split holds for the keyboard, the network interface, and the graphics adapter. Because each controller runs on its own, the CPU and the controllers are concurrent agents — and the bus is the shared resource they contend for.

> **Teaching note:** Figure 2.1 deliberately omits the interrupt controller. Do not let a sharp student conclude that the machine has no signalling hardware. Say plainly: "the anatomy is the data path; the next diagram adds the wire that lets a device get attention." The interrupt controller appears in Section 2.4, and it is the piece that turns this static picture into an event-driven system.

### Bootstrap: how the machine starts

Write the boot sequence in the top-left corner, vertically, before class if you can:

1. **Firmware** (BIOS or UEFI) runs from ROM or flash when power is applied. It tests the hardware with a power-on self-test (POST) and finds a boot device.
2. The **bootstrap program** loads the OS kernel from disk into memory and jumps to its entry point.
3. The kernel initializes its data structures, starts the first process (`init` or `systemd`), and from then on is **event-driven**.

The consequence is the sentence to say slowly: until the kernel is loaded, no OS exists; after it is loaded, the kernel only runs when a trap, an interrupt, or an explicit call gives it the CPU. That is why the rest of the lecture is about signals rather than about code.

**Board:** make the bootstrap a second drawable moment. Build order: draw a vertical three-box ladder (firmware → bootstrap → kernel), then add a downward arrow labelled "loads", then add a side arrow into the kernel labelled "trap / interrupt / call". Narration: "it boots in three steps, but it *lives* on this side arrow." Ask the room: "after boot, who is in charge when nothing is happening?" The correct answer — nobody; the CPU runs the idle loop until an event arrives — is the setup for the next section.

Concepts That Click

### The CPU's view: fetch, decode, execute — and wait

The CPU repeats one cycle: fetch an instruction at the program counter, decode it, execute it, update the PC. Its registers hold the working set: general-purpose registers, the program counter (PC), the stack pointer, and the status word.

The problem is speed. I/O is orders of magnitude slower than the CPU. A 3 GHz core executes roughly three instructions per nanosecond; a disk access takes milliseconds. If the CPU waited for every device, it would idle almost all the time. The OS therefore needs a way to *start* an I/O operation and be told later that it finished — without watching it. Every mechanism in the rest of the lecture is an answer to that sentence.

> **Teaching note:** The 3 GHz figure is worth saying out loud because it makes the mismatch visceral: in the time one disk access completes, the core could have executed millions of instructions. Ask students to estimate the ratio before you give it. They will guess hundreds, not millions, and the correction is memorable.

## 2.4 Controllers, Drivers, and the Interrupt Path

Every device has two faces, one in hardware and one in software.

**Definition (Device controller (hardware)).** *A chip or board with registers the CPU can read and write: **status** (busy, ready, error), **control** (start, reset, mode), and **data-in / data-out** buffers. It operates the physical device.*

**Definition (Device driver (software)).** *Kernel code that knows the controller's register layout and protocol. The OS calls the driver; the driver programs the controller; the controller moves the data.*

There is one driver per controller family, and the rest of the kernel sees a uniform interface (read, write, ioctl). The kernel also keeps a **device-status table** listing each device, its type, address, and state. The table matters later in the hour: when an interrupt arrives, the kernel looks up the device, updates its state, and wakes any process waiting on it.

Now walk the Enter trace. Keep the five steps numbered on the board, because the second half of the lecture reuses steps four and five.

1. You press Enter. The keyboard controller scans the key matrix and places a scan code in its data register.
2. The controller asserts an **interrupt request (IRQ)** line on the bus.
3. The CPU finishes the current instruction, saves its state, and jumps to the keyboard **interrupt service routine (ISR)**.
4. The ISR reads the scan code, maps it to a character, and queues it for the terminal process.
5. The scheduler eventually runs the shell, which reads the character and starts `cat bigfile.txt` — which now needs the disk.

The pattern to remember: device → controller → IRQ → CPU saves state → ISR → kernel decides what happens next. Every device in the machine uses a variant of this path.

**Board:** draw Figure 2.2 in four stages, not all at once. First the device box and the arrow leaving it, labelled "IRQ". Second the interrupt controller box, with the line "prioritizes and forwards". Third the CPU and the ISR boxes, with the arrow labelled "dispatch". Fourth, and only

Concepts That Click

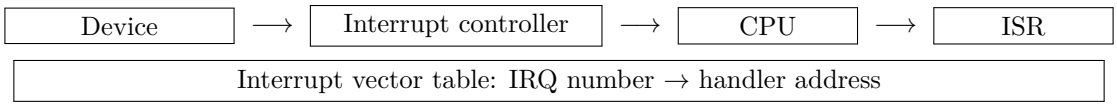



Figure 2.2: The interrupt path: device, interrupt controller, CPU, and interrupt service routine, dispatched through the vector table.

after the horizontal path is complete, drop a line from the CPU down to the vector-table box and label it "index by IRQ number". The order matters: if you draw the table first, students think the table is the thing that detects the device. Narrate each arrow with the verb it performs — asserts, prioritizes, dispatches, services.

**Board:** a second, cold-call example. Ask a student to redraw the same path for a *disk completion* rather than a key press, using the four-step interrupt-driven I/O sequence from Section 2.9. The path is identical; only the last step changes — the ISR acknowledges the controller, updates the device-status table, and wakes the waiting process. If the student puts the wake before the acknowledge, that is the moment to discuss why the device must be quieted before the kernel does anything slow.

> **Teaching note:** This is where to kill the "the CPU jumps immediately" misconception. Say the sentence with emphasis: the CPU finishes the *current instruction*, not the current program. On a pipelined machine even "current instruction" is a simplification, but for this course the atomic-instruction picture is the one to build on. Save the precise discussion for the architecture course.

## 2.5 Interrupts and Traps: Vector Table, Masking, Priority, and the Two Halves

There are two kinds of transfer of control into the kernel, and the difference is timing.

**Definition (Interrupt (asynchronous)).** *A transfer raised by* hardware *at an unpredictable time: a key press, a packet arrival, a disk completing a transfer. It is unrelated to the instruction the CPU is currently executing.*

**Definition (Trap or exception (synchronous)).** *A transfer raised by the* CPU *because of the instruction just executed: a system call, a divide by zero, a page fault, an illegal opcode. It is caused by the program itself.*

Both transfer control to a kernel handler and both save the CPU state. The distinction matters because an interrupt can arrive *between* any two instructions, whereas a trap is tied to a specific instruction.

**Board:** write five events in a column and have the room classify each before you mark it: key press, divide by zero, disk completion, page fault, packet arrival. (Add "system call" if the room is quick.) The expected split is interrupt, trap, interrupt, trap, interrupt, trap. Ask for the *reason* each time, not just the label; the reason is always "is it tied to the instruction just executed?" This takes ninety seconds and prevents a dozen exam errors.

The **interrupt vector table** is an array of handler addresses indexed by interrupt number. It lets the CPU dispatch without searching: the IRQ number itself is the index. **Maskable** interrupts can be temporarily disabled by the kernel while it updates shared structures; **non-maskable** interrupts (a power failure, a fatal hardware error) cannot. **Priority levels** decide which interrupt

Concepts That Click

wins when several arrive at once, and a low-priority handler can itself be interrupted by a high-priority one.

**Board:** the priority scenario. Draw two handlers as horizontal bars — keyboard (low) and disk (high) — and place a disk IRQ arrival in the middle of the keyboard bar while interrupts are enabled. The high-priority bar preempts; the keyboard bar resumes after. Then redraw with the keyboard handler having masked interrupts: the disk waits until the mask is lifted. Narration: "masking does not turn the device off; it postpones delivery." That sentence is the bridge to the missed-interrupt problem below.

There is a design tension here. Handlers must be short, or they block other interrupts and add latency. Real kernels therefore split the work: a minimal **top half** acknowledges the device, and a deferred **bottom half** does the rest (Linux softirqs, tasklets, and work queues).

Ask Socratic Check 2 now, before revealing the split: "An interrupt handler runs with interrupts temporarily disabled. The handler is long. What can go wrong, and what does this imply about handler design?" Give the room a minute. The answers you are listening for are: other devices cannot deliver, so their data can be overwritten or lost (a **missed interrupt**); and the system's **interrupt latency** grows until a real-time device may miss its deadline. Then the top-half/bottom-half split is not a fact to memorize but the obvious consequence: acknowledge quickly, defer the heavy work to a context where interrupts are enabled.

> **Teaching note:** Students often hear "maskable" as "optional" or "ignorable". Correct it with the physical picture: the device has already put the byte in its data register. Masking delays the CPU's response, and if a second byte arrives before the first is read, the first is gone. That is why the mask window is measured in instructions, not milliseconds.

Concepts That Click

## 2.6 Worked Example: What Interrupt Handling Costs

Setup, written on the board exactly as the deck states it: saving the context and entering the handler costs 2 $\mu$s; the handler itself runs for 8 $\mu$s. Each interrupt therefore costs 10 $\mu$s of CPU time.

**Board:** build the table one row at a time, converting units aloud. Do not write the percentages first; derive them so students see the cancellation interrupts/s $\times$ $\mu$s = $\mu$s/s.

| Rate (interrupts/s) | CPU time per second | Fraction | Overhead |
|---:|---:|---:|---:|
| 1,000 | 10 ms | 10/1000 | 1% |
| 10,000 | 100 ms | 100/1000 | 10% |
| 100,000 | 1,000 ms | 1000/1000 | 100% |

At 1,000 interrupts/s the machine loses one percent of its cycles; at 10,000 it loses a tenth; at 100,000 it spends all of its time entering and leaving handlers and does no useful work. The scaling is linear because the cost is per event, and that is the whole lesson: a device that produces a *stream* of events makes per-event interrupts untenable. DMA is the answer for block transfers, and Section 2.9 makes the comparison quantitative.

**Board:** the reverse question, which is where the idea sticks. "At this same 10 $\mu$s per interrupt, what rate consumes half the CPU?" Let them solve it: 50% of one second is 500 ms, so 500 ms/10 $\mu$s = 50,000 interrupts/s. Then ask the prediction trap: "A network card that interrupts once per 4 KB packet at 1 GB/s — is that above or below the halfway point?" Do not answer it; the arithmetic is the next lecture's business. The point is that the halfway point is a *rate*, not a byte count, so packet size is the lever.

**Teaching note:** Unit errors dominate this topic. Write 10 $\mu$s $= 10 \times 10^{-6}$ s once, explicitly, and insist that every row carry its unit. A student who writes "1,000 × 10 = 10,000" and then calls it 10 ms has the right idea and the wrong arithmetic; mark the units, not the concept. Also say "one percent of a 3 GHz core is still 30 million cycles per second" — cheap per event, not free.

## 2.7 Storage: Main Memory, Secondary Storage, and Disk Geometry

Interrupts tell the CPU that data has arrived. The next question is where the data live.

**Definition (Main memory (RAM)).** *A large array of bytes, each with an address, implemented in DRAM. It is fast (tens of nanoseconds) and directly addressable by the CPU, but it is volatile: its contents vanish when power is lost.*

**Definition (Secondary storage).** ***Hard disks** (magnetic platters) and **solid-state drives** (NAND flash). It is non-volatile and far larger than RAM, but orders of magnitude slower — microseconds for an SSD and milliseconds for a spinning disk.*

Programs live in memory only while they run; files persist on secondary storage. The OS moves data between the two in units of **blocks** (on disk) or **pages** (in memory). Keep those two words separate: a block is the device's transfer unit, a page is the memory manager's unit, and they need not be the same size.

A hard disk is organized around rotating **platters**. Each surface has concentric **tracks** split into **sectors**, and the same track across platters is a **cylinder**. Access time has three components: seek (move the arm to the track; mechanical, milliseconds), rotational latency (wait for the sector to pass under the head; milliseconds), and transfer (stream the requested sectors; microseconds per block).

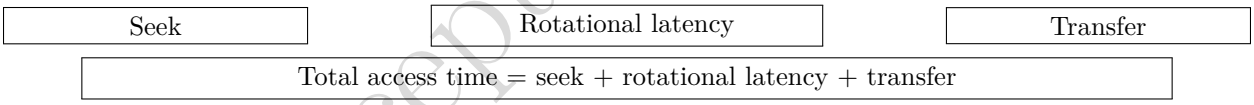



Figure 2.3: The three components of disk access time. The first two are mechanical and dominate.

**Board:** draw Figure 2.3 as a horizontal bar, not as boxes: a long segment for seek, a similar segment for rotational latency, and a visually tiny sliver for transfer. The picture is the argument — the sliver is why disk scheduling is about *order*, not bandwidth. Then write the four given parameters in a corner: seek 8 ms, 7,200 RPM, 150 MB/s, block 4 KB.

One rotation takes 60/7,200 s = 8.33 ms, so the average rotational latency is half of that, 4.17 ms — on average the sector is halfway around. The transfer time is 4 KB/150 MB/s $\approx$ 27 $\mu$s. The total is therefore

$$8 + 4.17 + 0.027 \approx 12.2 \text{ ms},$$

of which 99.8% is mechanical waiting.

**Board:** second example, the sensitivity check. Ask first: "If I double the transfer rate to 300 MB/s, what happens to the total?" Most rooms predict a large improvement. Compute: transfer becomes about 13 $\mu$s, so the total is $8 + 4.17 + 0.013 \approx 12.18$ ms — essentially unchanged. Even an infinitely fast transfer leaves 12.17 ms of seek and rotation. This is the cleanest way to show that the mechanical terms, not the streaming term, set the floor.

Concepts That Click

The consequence for the OS is that disk performance is dominated by **where the head is**, not by how fast bytes stream. This is why scheduling the *order* of requests (disk scheduling, later in the course) matters so much.

> **Teaching note:** Derive the half-rotation rule rather than assert it. "One rotation is 8.33 ms; if the sector could be anywhere on the track with equal probability, the average wait is half a rotation." Students accept 4.17 much more readily when it is a symmetry argument rather than a memorized formula. Keep the units in milliseconds and microseconds separate; the 0.027 ms term is the one that gets dropped by mistake.

## 2.8 The Memory Hierarchy, Locality, and Caching

Storage is not one technology but a hierarchy: registers, cache, main memory, solid-state drive, magnetic disk, and tape or optical archive.

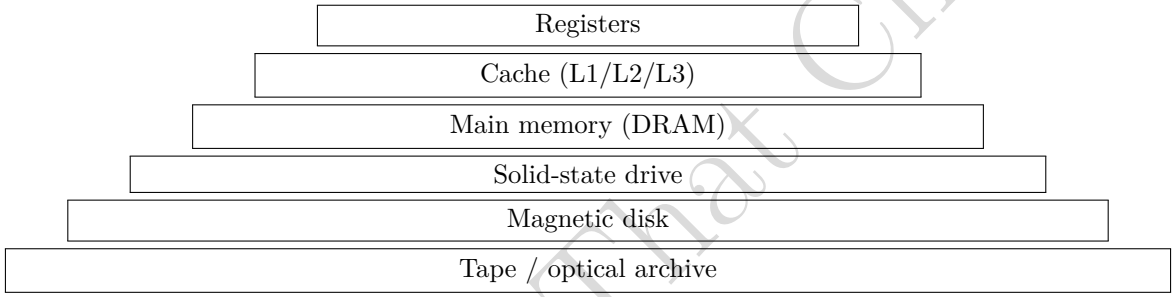



Figure 2.4: The memory hierarchy: capacity grows and cost per byte falls as access time grows.

Moving down the hierarchy, capacity grows, cost per byte falls, and access time grows by orders of magnitude. Moving up, data is **cached**: a copy is kept in the faster level on the bet that it will be used again. That bet is the entire reason the hierarchy works.

**Board:** draw the ladder as a set of centred bars of increasing width, as in Figure 2.4, and label the two axes verbally: width is capacity, height is speed. Do not put cost on the same axis; students conflate "cheap per byte" with "fast". Then ask: "Which level is the only one the CPU can name directly?" Answer: registers and memory, by address; cache is invisible to the instruction set and managed by hardware. That distinction sets up the buffer cache, which *is* managed by the OS.

The hierarchy works because programs do not access memory uniformly. **Temporal locality**: an item used once is likely to be used again soon. **Spatial locality**: an item near one just used is likely to be used soon.

**Definition (Cache).** *A small, fast store holding copies of recently used data. A **hit** is served from the cache; a **miss** goes to the next level and installs a copy.*

The OS manages caches at every level: CPU caches (in hardware), the **buffer cache** for disk blocks, the TLB for page translations, and memory itself as a cache for disk.

**Board:** second example, a read trace. Write "`read()` block $B$" and branch: buffer-cache hit (return the copy; no device I/O) or miss (start device transfer, sleep the process, DMA installs the block, wake the process). This is the deck's thread-in-one-picture compressed to two branches, and it lets you point back at the interrupt path: the wake happens from the ISR, through the device-status table. Ask the room to name which branch costs an interrupt.

Ask Socratic Check 3 here: "Main memory is volatile, yet a file you saved last week still exists after a power cut. Where is the file, and what role did RAM play in saving it?" The file lives on

Concepts That Click

secondary storage, which is non-volatile; RAM held the program's buffer and the kernel's cache while the write was in progress, and the OS copied the dirty blocks to storage. This is why "save" is an I/O operation, not a memory operation — and why a crash before the write-back can lose data.

> **Teaching note:** The sentence to leave in the room: a cache is a *copy*, never the authoritative home of the data. Students who think "the file is in RAM" will later be confused by write-back, crash consistency, and why `fsync` exists. Also distinguish the hardware-managed CPU cache from the OS-managed buffer cache; both are caches, but only one is a policy the kernel chooses.

## 2.9 I/O Structures: Polling, Interrupts, and DMA

Data must come from a device into memory. The remaining question is who moves it: the CPU, one word at a time, or a controller acting on its own. The three techniques form a progression.

### Polling

**Definition (Programmed I/O (polling)).** *The CPU repeatedly reads the controller's* ***status register*** *until the device is ready, then moves the data word by word.*

Polling is simple: no interrupt machinery and no concurrency. It is also wasteful: every poll is a bus transaction and a wasted instruction slot, and a slow device wastes almost all of them. It is still the right choice for very fast devices (a GPU register) or when the wait is predictably short — polling a device that is always ready beats paying interrupt overhead.

This is also the answer to Socratic Check 1, which you should pose before giving it: "Why can the OS not simply poll every device in a loop? Name two distinct costs." The two costs are **CPU cost** (cycles that could run user code, and most polls find nothing ready) and **latency cost** (a slow or sporadic device may wait an entire polling period before being noticed). Interrupts invert the arrangement: the device tells the CPU when it needs attention.

### Interrupt-driven I/O

The driver starts the device by writing its control register; the CPU returns to other work and the scheduler runs another process; when the device finishes it raises an interrupt; the ISR reads the data or a status, acknowledges the device, and wakes the waiting process. The cost model is the key: CPU cost scales with the *number of events*, not the number of bytes. Small blocks mean many interrupts; large blocks mean few. This is the lever DMA pulls.

### Direct memory access

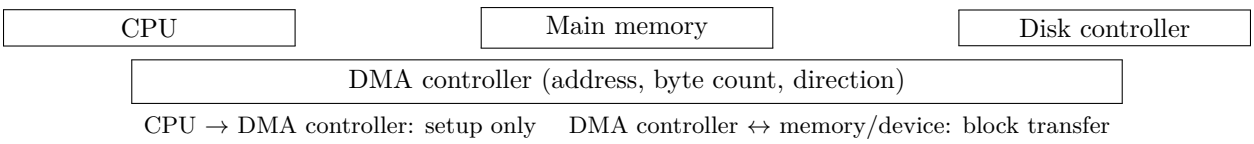



Figure 2.5: Direct memory access: the CPU sets up the transfer, and the controller moves the block.

Concepts That Click

**Board:** draw Figure 2.5 in three stages. First the three boxes CPU, memory, disk controller. Second the DMA controller underneath, with the dashed "setup" arrow from the CPU only. Third the two-way block-transfer arrows from the DMA controller to memory and to the disk controller. Narrate: "the CPU writes three things — address, count, direction — and leaves; the controller does the moving; one interrupt at the end." Emphasize that the dashed arrow is the *only* CPU involvement after setup.

DMA does not mean the transfer uses no CPU time at all. The hidden costs are **setup** (the CPU programs address, count, and direction for every transfer), **bus contention** (the DMA controller steals memory cycles, slowing the CPU's own memory accesses — cycle stealing), and **cache coherence** (the CPU may hold stale copies of memory the DMA wrote, so the OS and hardware must invalidate or flush). DMA is a *trade* of per-byte CPU work for per-block setup and bus pressure, not a free lunch. That is Socratic Check 4, and it should be asked before the list is revealed.

### Worked example: moving 1 MB three ways

Parameters: disk transfer rate 200 MB/s; interrupt cost 10 $\mu$s; DMA setup plus completion interrupt about 20 $\mu$s; transfer time for 1 MB is 5 ms.

| Technique | Blocks | Interrupt cost | Share of 5 ms |
|---|---|---|---|
| Interrupt-driven | 4 KB (256 interrupts) | 2.56 ms | 51% |
| Interrupt-driven | 64 KB (16 interrupts) | 160 $\mu$s | 3.2% |
| DMA | one block | 20 $\mu$s | 0.4% |

**Board:** build the arithmetic in three columns and let the room do the division: 1 MB/4 KB = 256; 256 × 10 $\mu$s = 2.56 ms; 2.56/5 = 51%. Then 1 MB/64 KB = 16; 16 × 10 $\mu$s = 160 $\mu$s; 160/5000 = 3.2%. Then the DMA row: 20 $\mu$s, about 0.4%, with the CPU free to run other work.

**Board:** second example, the block-size sweep. "Same 1 MB, same 10 $\mu$s per interrupt, but 32 KB blocks." 1 MB/32 KB = 32 interrupts; 32 × 10 $\mu$s = 320 $\mu$s; 320/5000 = 6.4%. The point is not the number but the shape: halving the block size doubles the interrupt count and the overhead. Then ask the decision question: "One 4 KB block only — interrupt or DMA?" At these parameters interrupt-driven costs 10 $\mu$s and DMA setup costs about 20 $\mu$s, so the interrupt wins for a single small block. DMA wins on *large* transfers, not on a single byte. That is the deck's trade-off stated precisely.

> **Teaching note:** Do not let the table become a memorized ranking. The ranking depends on the parameters. The invariant is the shape: interrupt cost is per event, DMA cost is per transfer, so there is always a crossover. If a student asks "which is faster", the correct reply is "at what block size?". The one 4 KB decision is worth two minutes because it forces them to read the parameters instead of the table.

### Device types and the device-status table

**Block devices** are addressable in fixed-size blocks and are seekable — disks, SSDs, USB drives. They support file systems and DMA. **Character devices** deliver a stream of bytes and are not seekable — keyboards, serial ports, mice. They are naturally interrupt-driven. The kernel maintains the **device-status table**: for each device, its type, address, and current state (idle, busy, error). When an interrupt arrives, the kernel looks up the device, updates its state, and wakes any process waiting on it.

Concepts That Click

The progression, in one sentence: the CPU does *more* waiting under polling, *less* work under interrupt-driven I/O, and *no* per-byte work under DMA. `cat bigfile.txt` uses a block device, so DMA is the natural choice; a keyboard uses interrupt-driven I/O.

## 2.10 Computer-System Architecture: SMP, Multicore, NUMA, and Clusters

One CPU, one memory, one disk is the simple case. Real machines replicate processors and whole systems, and the OS must keep them coherent.

**Definition (Single-processor system).** *A system with one general-purpose CPU. Most personal computers and phones have several cores per chip, but the OS still sees a set of processors to schedule.*

A **multiprocessor** system has two or more processors that share the bus, clock, and memory. In **symmetric multiprocessing (SMP)**, every processor is equal and runs the same OS, so the scheduler can place any task on any CPU. In an **asymmetric** system, one master processor assigns work to slaves — simpler, but the master is a bottleneck. **Multicore** is a different packaging of the same idea: several independent cores on one chip, each with its own registers and usually its own L1/L2 cache, sharing a last-level cache and memory.

A **cluster** goes further: two or more complete systems connected by a network, sharing storage and presenting as one service. Clusters are used for **high availability** (failover) and **high-performance computing** (parallel work). **NUMA (non-uniform memory access)** describes large multiprocessors in which memory is attached to nodes: a core accesses local memory faster than a remote node's memory, so the scheduler and allocator should keep a task near its data.

**Board:** draw the three-column comparison as the lecture's last table.

| | **SMP** | **Multicore** | **Cluster** |
|---|---|---|---|
| What is shared? | Bus, clock, memory | Last-level cache, memory | Network, storage |
| Failure unit | One processor | One core | One whole machine |
| Main benefit | Throughput, scaling | Throughput, lower latency | Availability, scale-out |
| OS challenge | Scheduling, locking | Cache affinity, locking | Distributed coordination |

Narration while drawing: all three give **parallelism**; only a cluster survives the failure of an entire machine; all three make concurrency real for the OS, which is Unit II. Ask the room to fill the "failure unit" row before you write it; that row is the one they remember.

**Board:** the four-core, single-bus discussion, which is Socratic Check 5. Draw four core boxes on top of one narrow bus, with two memory arrays below. "Two programs each read a 1 GB array. What limits the speedup?" Let them argue for cores, cache, and clock before you say: the **memory bus** is the shared bottleneck; more cores do not add memory bandwidth, so the workload is **memory-bound**, not CPU-bound. First fixes: improve **locality** (block the loops so each cache line is reused), then add cache or bandwidth — not more cores. This is why the OS and compiler care about caches and NUMA placement, not just the processor count.

> **Teaching note:** The four-core question is the one students will repeat in an exam, so give them the vocabulary, not just the answer: *memory-bound* versus *CPU-bound*. A useful follow-up: "Would adding a second bus help?" It can, but then cache coherence and NUMA placement become the next bottleneck — which is exactly the recurring theme: every layer of hardware organization becomes a policy question for the OS. Do not spend more than five minutes here; the detailed treatment belongs to the multiprocessor scheduling lecture.

Concepts That Click

## 2.11 Where Students Get Stuck

- **"An interrupt and a trap are the same thing."** They share a destination (a kernel handler) and a save-state step, but not a cause. An interrupt is asynchronous hardware; a trap is synchronous, caused by the instruction just executed. Force the test: "is it tied to this instruction?" If yes, trap.
- **"The CPU abandons the instruction it is running."** It finishes the current instruction, saves the PC and status word, and only then dispatches. This is the single most common misstatement in the room; correct it at the diagram, not later.
- **"DMA means zero CPU cost."** Setup, bus contention (cycle stealing), and cache coherence are all CPU or system costs. DMA removes per-byte work, not all work.
- **"A cache is extra memory."** A cache is a *copy* of data that lives authoritatively somewhere else. The buffer cache holds disk blocks; the disk still owns the file.
- **"Volatile means the file disappears when the program exits."** Volatile means power loss, and it applies to RAM, not to the file on secondary storage. "Save" is an I/O operation, not a memory operation.
- **"Polling is always the wrong design."** Polling is right for a fast, always-ready device and for predictably short waits. The weakness is busy-waiting on slow or sporadic devices, not polling as such.
- **"Four cores give four times the speed."** Only if the workload is CPU-bound and the shared bus, cache, and memory can feed all four. The 1 GB array example is memory-bound and will not scale that way.
- **"The handler does all the work."** The top half acknowledges; the bottom half does the heavy work with interrupts enabled. Conflating the two is what produces the "long handler" error in Socratic Check 2.
- **"The interrupt vector table stores data."** It stores *handler addresses*, indexed by interrupt number. Its purpose is dispatch without searching.

**Teaching note:** Do not read this list aloud as a list. Each item should surface at the diagram or calculation where it belongs — that is why the asides above are placed where they are. This section is your post-class checklist: if the room left without hearing items 1, 2, and 3 corrected, the lecture did not land.

Concepts That Click

## 2.12 Board Plan and Timing

### 75-minute sequence

| Minutes | Activity |
|---|---|
| 0–5 | Opening: write `cat bigfile.txt`; collect components; pose "who can signal the CPU?" |
| 5–13 | Anatomy (draw bus first) and bootstrap ladder; CPU fetch/decode/execute and the utilization problem |
| 13–22 | Controller vs. driver; four registers; device-status table; Enter trace (five steps on the board) |
| 22–31 | Interrupt vs. trap (classify five events); interrupt path diagram; vector table, masking, priority |
| 31–39 | Interrupt-cost worked example; reverse solve for the 50% rate; top-half/bottom-half |
| 39–48 | RAM vs. secondary; disk geometry; 4 KB/7,200 RPM worked example and sensitivity check |
| 48–56 | Memory hierarchy; locality; cache hit/miss; read trace; Socratic Check 3 |
| 56–67 | Polling, interrupt-driven, DMA (draw DMA in three stages); 1 MB table; 32 KB sweep; hidden costs; device types |
| 67–73 | SMP, multicore, NUMA, clusters; comparison table; four-core single-bus discussion |
| 73–75 | Exit ticket (five Socratic checks, one-minute paper) |

### 50-minute compression

| Minutes | Activity |
|---|---|
| 0–4 | Opening and roadmap |
| 4–10 | Anatomy and bootstrap |
| 10–17 | Controllers, drivers, Enter trace, interrupt path |
| 17–24 | Interrupts vs. traps; vector, masking, priority; top/bottom half |
| 24–29 | Interrupt-cost example (give the 100% row as a result) |
| 29–37 | Storage, disk geometry, worked example |
| 37–43 | Hierarchy, locality, caching |
| 43–48 | I/O three ways and DMA; give the 1 MB table as a summary |
| 48–50 | Architecture comparison and exit ticket |

**Teaching note:** If you fall behind, cut the cluster comparison and the sensitivity check, and compress the hierarchy to the ladder plus the save-file answer. Never cut the interrupt path, the interrupt-cost example, or the disk computation: those three are the assessed core. Write the minute marks on your own copy, not on the board — a visible clock makes students anxious and makes you rush the good parts.

Concepts That Click

## 2.13 Exit Ticket and Comprehension Checks

Use the deck's five Socratic Checks in order. Each takes under a minute if you take one answer and correct it.

1. **Polling.** "Why can the OS not poll every device in a loop? Name two distinct costs." Listen for CPU cost and latency cost; reject a single vague "it is slow".

2. **Long handler.** "An interrupt handler runs with interrupts disabled and is long. What goes wrong?" Listen for missed interrupts and growing latency, then for the top/bottom split.

3. **Volatile RAM.** "Where is the saved file, and what did RAM do?" Listen for secondary storage plus buffer/cache, and for "save is I/O".

4. **DMA cost.** "Does DMA use no CPU time? Name the hidden costs." Listen for setup, bus contention/cycle stealing, cache coherence.

5. **Four cores, one bus.** "What limits the speedup, and what do you change first?" Listen for memory-bound, then locality before more cores.

Close with a one-minute paper, collected or not: "Write one thing that signals the CPU, one thing that moves bytes without the CPU, and one number from today that surprised you." The third item tells you which example landed. If time allows, assign the 8 KB variant of the 1 MB table (128 interrupts, 1.28 ms, 25.6%) as a two-minute exit exercise; it is the same arithmetic and it exposes anyone who memorized the table instead of the model.

## 2.14 Readings to Prepare Next Time

Next lecture is about how the OS *stays in control*: dual-mode operation, the timer, privileged instructions, and the protection boundary that makes every mechanism in this lecture safe to share.

- **Silberschatz et al., *Operating System Concepts*, 8e, Ch. 1 §1.5 (pp. 20–23)** — operating-system operations: dual-mode operation, the mode bit, the timer, and why the kernel must be able to regain the CPU. *(Page-verified.)* This is the primary reading; the timer is the piece students forget.
- **Stallings, *Operating Systems: Internals and Design Principles*, 6e, Ch. 3 §3.5–3.6 (p. 140, p. 143)** — execution of the OS and the security issues that motivate protection. *(Page-verified.)* Read §3.5 for the kernel/user distinction in Stallings' vocabulary.
- **Optional: re-read Silberschatz Ch. 1 §1.2 (pp. 6–18)** with the next lecture in mind, noting every place the hardware must be able to interrupt or restrict the CPU. That list becomes the protection lecture's outline.
- **Optional concrete depth: Bovet & Cesati, *Understanding the Linux Kernel*, 3e, Ch. 4 (pp. 131–188)** — how Linux enters and leaves an interrupt, if you want to answer "what does the code actually do?". *(Page-verified.)*

Bring one question to the next class: *how does the kernel stop a user process that never makes a system call and never faults?* The answer is the timer, and it is the hinge between this lecture and the next.

Concepts That Click